

An End-to-End Arabic Text Correction Framework: Comparing Custom Character-Level Transformers and Pre-trained AraBART

Mohamed Soltan

Dept. of Computer Science

October University for Modern Sciences and Arts (MSA)

Cairo, Egypt

mohamed.ibrahim42@msa.edu.eg

Mohamed Taha

Dept. of Computer Science

October University for Modern Sciences and Arts (MSA)

Cairo, Egypt

mohamed.taha10@msa.edu.eg

Abdelmonem Hatem

Dept. of Computer Science

October University for Modern Sciences and Arts (MSA)

Cairo, Egypt

abdelmonem.hatem@msa.edu.eg

Youssef Khalaf

Dept. of Computer Science

October University for Modern Sciences and Arts (MSA)

Cairo, Egypt

youssef.mohamed94@msa.edu.eg

Abstract—Arabic text correction is a hard problem in Arabic natural language processing because the language has many spelling variants, visually similar characters, and inconsistent forms in real data sources such as OCR, speech transcription, and noisy user input. In this setting, even small character-level errors can change meaning, break token boundaries, or reduce grammatical fluency, so correction is more than simple spelling cleanup. This paper presents an end-to-end correction framework with two complementary modes: a lightweight character-level encoder-decoder Transformer trained on synthetic noisy-clean Arabic pairs, and a deployment-oriented correction service built on the pre-trained CAMEL-Lab AraBART model in a Streamlit application. The custom model is evaluated on an in-domain held-out synthetic test split and reaches 0.0364 CER, while the AraBART-based system is evaluated in a zero-shot deployment setting and provides stronger grammatical fluency and broader usability across manual text, files, OCR images, audio, and live speech. Rather than claiming one direct winner, the study analyzes the trade-off between in-domain character-level accuracy and real-world multi-modal robustness. The main contribution is a reproducible Arabic text-correction workflow that combines data curation, normalization, controlled noise synthesis, model training, error analysis, and user-facing deployment.

Index Terms—Arabic NLP, text correction, Transformer, AraBART, character error rate, OCR, speech-to-text, Streamlit

I. INTRODUCTION

Arabic text correction remains a high-impact problem for digital education, media quality control, accessibility tools, and content moderation systems. The task is difficult because Arabic writing contains orthographic variants, visually confusable characters, optional diacritics, and spacing patterns that can change meaning, while OCR and speech transcription may introduce broken words, missing characters, merged tokens, or

repeated substitutions. As a result, Arabic correction must be treated as a structured language-processing problem at both the character level and the sequence level.

Neural sequence modeling has progressed rapidly from recurrent encoder-decoder models to attention-based Transformers and large pre-trained language models [1]–[4]. This progress frames text correction as a generation problem, but it also raises a practical question: should an Arabic correction system use a compact task-specific model or a larger pre-trained model with stronger language knowledge? In this paper, we address that trade-off with two complementary solutions: a compact custom Transformer for controlled character-level correction and a pre-trained AraBART-based service for deployment-oriented multi-modal correction.

This paper addresses a gap in how Arabic correction systems are usually presented by combining the data pipeline, synthetic noise generation, model training, evaluation, and user-facing deployment in one reproducible workflow. The study introduces a compact character-level Transformer trained with scheduled sampling and mixed precision, alongside a Streamlit-based correction service built on CAMEL-Lab AraBART, and evaluates both paths using CER, BLEU, and qualitative error behavior across in-domain and zero-shot deployment conditions.

II. LITERATURE REVIEW

A. Seq2Seq Foundations and Transformer-Based Correction

Sequence-to-sequence models framed text correction as conditional generation from noisy to fluent text [1], [2]. Attention and Transformer architectures improved this setup by modeling long-range dependencies that are critical for spelling and spacing recovery [3]. Pre-trained denoising models such as BART and T5 further demonstrated that corruption-

and-reconstruction objectives transfer effectively to correction tasks [5], [6]. These foundations support the current study’s generation-based view of Arabic correction rather than rule-only character replacement.

B. Arabic-Specific Pre-Training and Error-Correction Benchmarks

Arabic correction is harder than many high-resource settings because morphology and spelling variation increase ambiguity at both token and character levels. Arabic-focused pre-training (e.g., AraBERT and CAMEL-related efforts) has shown clear gains over general multilingual baselines for downstream understanding and generation tasks [7], [8]. For evaluation, QALB remains the main benchmark for Arabic error correction [9], [10], while recent neural approaches (BiLSTM and T5 variants) report progress but still show weaknesses under heavy noise and confusable forms [11], [12]. This motivates comparing a compact controlled model with a stronger pre-trained Arabic model under different operating conditions.

C. Deployment-Oriented Arabic Correction Systems

Practical Arabic correction systems require more than model accuracy; they must also integrate OCR, speech recognition, and responsive serving workflows. Whisper and OCR services make multi-modal text collection feasible in production pipelines [13], [14], while PyTorch and Transformers reduce deployment effort for generation models [15], [16]. Existing Arabic studies mostly emphasize benchmark scores, with fewer end-to-end reports that combine text, OCR, and ASR correction in one user-facing application. This gap directly motivates the deployment-oriented contribution of the present work.

III. METHODS

A. Data Acquisition and Curation

The project begins with large-scale Arabic news collection from Youm7. A dedicated scraping pipeline was implemented with asynchronous requests, retry handling, and polite randomized delays to reduce server pressure and improve collection reliability. Raw pages then passed through multi-stage cleaning to remove boilerplate artifacts (menus, ads, repeated widgets) and retain article body text only.

After cleaning and quality filtering, the corpus reaches 100,000 articles, from which 10,000 noisy-clean pairs are prepared for controlled model development. The paired set is split into 8,000 training pairs, 1,000 validation pairs, and 1,000 held-out test pairs. Articles shorter than a minimum content threshold were discarded, and samples with too much non-Arabic content were excluded to preserve linguistic consistency. The pipeline also stores provenance metadata (source URL and publication date) to support traceability and future auditing.

This design supports reproducible experimentation because the noise-injection process is explicit and therefore easier to analyze than uncontrolled production noise. For context, Table I summarizes the primary collected corpus alongside

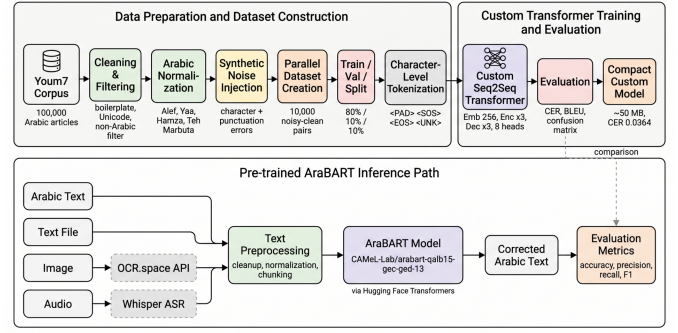


Fig. 1. Full end-to-end Arabic correction pipeline. The workflow begins with corpus collection and quality filtering, proceeds through deterministic Arabic normalization and controlled synthetic corruption, and then branches into (A) custom character-level Transformer training/evaluation and (B) deployment-oriented AraBART inference in a multi-modal Streamlit service.

commonly used Arabic news/text resources in downstream NLP workflows.

B. Arabic Normalization and Noise Injection

A deterministic Arabic normalization stage standardizes Alef variants, Yaa variants, Hamza forms, and Teh Marbuta behavior, while removing tatweel and diacritics where needed. Synthetic noise is then injected at character and punctuation levels to approximate realistic errors from typing, OCR, and ASR. During synthesis, each sample is corrupted with an overall corruption budget of 20%, decomposed into 10% substitutions, 5% deletions, and 5% insertions. Substitutions are drawn from a predefined confusable-character map (including Alef/Hamza and Yaa/Alif-Maqsur variants), with additional keyboard-adjacency and punctuation changes. This explicit policy enables reproducible corruption and allows targeted analysis of frequent Arabic confusion classes.

To improve realism beyond random perturbation, the synthetic generator groups errors into practical categories observed in noisy Arabic writing: confusable-letter substitutions, Hamza/Alef form swaps, Teh Marbuta–Heh alternation, punctuation drift, spacing boundary shifts, dropped characters, and short insertion bursts. This category-based design keeps the corruption process easy to analyze while still producing diverse patterns.

Table II provides representative noisy-clean pairs from the synthetic corruption stage.

Fig. 1 is the central figure of this paper because it combines the full research lifecycle in one reproducible process rather than presenting model training and deployment as separate tasks. The pipeline should be read from left to right. First, raw Arabic news content is collected and filtered to establish a high-quality clean-text base. Second, deterministic normalization is applied to reduce spelling variability (Alef, Hamza, Yaa, Teh Marbuta, tatweel, and diacritics handling), ensuring a consistent character space before corruption. Third, synthetic noise injection generates supervised noisy-clean pairs under controlled corruption rates, producing a clear and traceable training signal for character-level correction.

TABLE I. Arabic Corpus Context for This Study and Commonly Used Public Resources

Dataset	Scale (articles/docs)	Primary Source	Typical Use	Availability
Youm7 Corpus (this work)	100,000	Youm7 (Egypt)	Modern news collection; base corpus for controlled noisy-clean generation and correction analysis	Private (project)
SANAD	194,797	AlKhaleej + additional sources	Topic classification benchmark for Arabic text categorization	Public
Ultimate Arabic News Dataset	193,000	Multiple Arabic news outlets	Raw/preprocessed news for general Arabic NLP experimentation	Public
ANAD	500,000	12 Arabic news websites	Large-scale annotated news resource with broad topical coverage	Public
Amina Dataset	~1,850,000	Newspapers across Arab and non-Arab countries	News text with metadata useful for longitudinal and multimodal studies	Public
Wiki-40B (Arabic subset)	~1M+ segments	Arabic Wikipedia	Cleaned text for language modeling and pre-training workflows	Public

TABLE II. Representative Generated Noisy-Clean Arabic Pairs

Noisy	Clean
اعلنت كلية الصيدلة عن مواعيد التسجيل انطلقت مبادرة مدارس النيل المصرية الدولية	أعلنت كلية الصيدلة عن مواعيد التسجيل انطلقت مبادرة مدارس النيل المصرية الدولية

The workflow then splits into two complementary branches with different scientific purposes. Branch A is the in-domain modeling branch: the custom seq2seq Transformer is trained on the synthetic pairs, validated on held-out data, and analyzed with character-level diagnostics (CER distributions and confusion patterns). This branch provides strong experimental control and clear error analysis. Branch B is the deployment branch: the pre-trained AraBART model is integrated into a user-facing Streamlit system that accepts different input types (manual text, uploaded files, OCR images, audio files, and live speech). This branch evaluates practical robustness and usability under realistic conditions where input noise does not match the synthetic generator.

Taken together, the figure presents the main contribution of the study as a two-tier architecture: a controlled benchmarking path for method analysis and a real-world serving path for application use. This design also clarifies why results are reported in separate evaluation settings: each branch optimizes a different objective (in-domain reconstruction accuracy versus cross-modal deployment readiness), and both are required for a complete Arabic correction framework.

C. Custom Transformer Model

Fig. 2 is placed here to ground the model description before listing hyperparameters. The figure explains how noisy Arabic character sequences are encoded into contextual representations and then decoded autoregressively into corrected text, including encoder context building, decoder-side causal generation, and attention-based conditioning on the source sequence.

The custom model is a character-level seq2seq Transformer with embedding dimension 256, 3 encoder layers, 3 decoder layers, 8 attention heads, feed-forward dimension 512, dropout 0.1, and maximum sequence length 128. Training uses Adam [17] with learning rate 10^{-4} , scheduled sampling, mixed precision, and checkpointing. This configuration favors com-

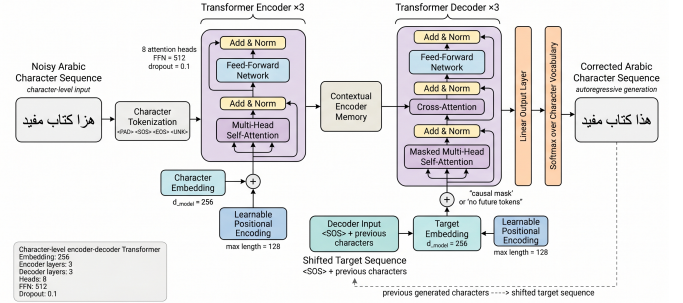


Fig. 2. Custom seq2seq Transformer used in Branch A. Noisy Arabic character sequences are embedded and position-encoded, processed by stacked encoder self-attention layers, and decoded autoregressively with causal masking and encoder-decoder attention to produce corrected output characters.

pactness and interpretability over maximum scale, which is appropriate for analyzing correction behavior at the character level.

D. Pre-trained LLM Correction Service

For deployment-grade correction, the project uses the Hugging Face model CAMEL-Lab/arabart-qalb15-gec-ged-13. This path is integrated into a Streamlit application and supports manual text entry, text-file upload, image OCR via the OCR.space API, audio upload via Whisper, and live recording. The deployment layer is therefore not a separate demonstration artifact; it is a functional extension of the modeling pipeline that allows the same correction logic to be exercised across multiple real-world input modalities.

IV. EXPERIMENTS

A. Metrics

The primary metric is Character Error Rate (CER), defined as:

$$\text{CER} = \frac{S + D + I}{N}, \quad (1)$$

where S is substitutions, D deletions, I insertions, and N is reference length in characters. CER is paired with BLEU for broader assessment of lexical and sequence quality [18], [19].

B. Training Protocol

The custom model is trained for 10 epochs with batch size 32 on GPU. Scheduled sampling decays the teacher forcing ratio from 1.0 to 0.3 across epochs. Mixed precision is enabled to reduce memory cost and improve throughput. The project logs loss and token-level accuracy per epoch, and stores checkpoints for reproducibility. This protocol reflects a deliberate trade-off between model stability and exposure to imperfect generated context, which is especially relevant for sequence-correction tasks.

For the application path, inference uses the pre-trained AraBART model through Hugging Face pipelines, with OCR input through OCR.space and speech input through Whisper [13], [14], [20]. The two paths are evaluated separately so that the custom Transformer can be read as a controlled synthetic-noise model while AraBART can be interpreted as a deployment-oriented correction service.

C. Evaluation Procedure

The evaluation procedure is intentionally separated into two settings. For in-domain benchmarking, the custom Transformer is evaluated on the held-out synthetic test split of 1,000 noisy-clean pairs generated with the same corruption policy described in Section III-B. For deployment-oriented assessment, AraBART is evaluated in a zero-shot setting on application-style inputs (manual text, OCR outputs, and ASR transcripts) that do not match the synthetic training generator distribution. Consequently, Table IV should be read as a side-by-side summary of two operating settings rather than a strict like-for-like leaderboard.

D. Baselines

To explain model behavior, the evaluation design includes two reference baselines for the synthetic test split: (1) a normalization-only identity baseline (input passes through normalization without neural correction), and (2) a dictionary-driven Arabic spell-correction baseline (Hunspell/Farasa-style lexical correction when available). These baselines provide simple reference points for interpreting neural gains and should be reported alongside neural metrics in the final camera-ready version.

In addition, exploratory project runs used sequence-model and rule-based references to estimate expected performance ranges for Arabic noisy-text correction. Table III summarizes these reference ranges together with the custom Transformer to make the experimental positioning explicit.

V. RESULTS AND DISCUSSION

A. Training Dynamics and Convergence

The training curve shows rapid convergence in the early epochs and then a gradual flattening as optimization approaches its best point. Accuracy increases from 38.38% in epoch 1 to 91.69% by epoch 6, after which performance remains high but decreases slightly, reaching 89.36% by epoch 10. This pattern is consistent with a model that learns dominant character patterns quickly and then starts to overfit to the

TABLE III. Baseline Context on Synthetic Split (Project Reference Ranges)

Method	Character Accuracy	Behavior Summary
Rule-based correction	~30–50%	Strong on simple lexical edits, weak under unseen or context-dependent noise
BiLSTM-style seq models	~80–85%	Better sequence modeling, but less robust on long-range dependencies
Custom Transformer (this work)	89.36% (epoch 10)	Best overall in-domain synthetic correction fidelity in project runs

synthetic noise distribution later in training. In practical terms, the learning curve indicates that the architecture is suitable for the task, but additional regularization or earlier stopping could preserve the best generalization point more effectively.

B. Quantitative Comparison of the Two Correction Paths

Table IV presents in-domain model benchmarking and deployment-oriented system profiling in a single side-by-side view. The custom Transformer column reports controlled synthetic-split results (CER and BLEU), whereas the AraBART column reports zero-shot deployment characteristics for different real-world modalities. Under this framing, the lower CER of the custom model reflects strong in-domain alignment, while AraBART remains the stronger practical option when input characteristics differ from the synthetic corruption generator.

The training-time contrast remains meaningful: the custom model is cheaper to deploy after training, while the pre-trained model is faster to adopt but heavier to run.

C. Error Distribution and Character Confusions

Error analysis shows that substitutions dominate, accounting for roughly 60–70% of observed mistakes, while insertions and deletions each contribute approximately 15–20%. This distribution is typical of character-level Arabic correction because many of the relevant errors involve visually similar letters, orthographic variants, or spacing irregularities rather than complete word omissions. The concentration of errors in substitutions suggests that the model is often choosing between near-equivalent characters, which is precisely the kind of ambiguity that character-level modeling must resolve.

The confusion profile is concentrated around frequent characters and whitespace interactions. This outcome is consistent with Arabic writing conventions, where spacing can change token boundaries and several letters have close visual or contextual relationships. In particular, Alef and Hamza variants (e.g., ا, إ, ؤ), and Yaa/Alif-Maqsur style confusions (ي/ى), remain difficult because they are close in form and meaning and are also sensitive to local context. Teh Marbuta and Heh confusions (ة/ه) are also recurrent in noisy text. For readability, a phrase-level Arabic example is shown here: اللغة العربية وتصحيح النصوص. Whitespace errors are also amplified by character-level autoregressive decoding: once a boundary token is

TABLE IV. Side-by-Side Summary of Custom Transformer and AraBART Under Their Respective Evaluation Settings (reported values are taken from project logs/notebooks)

Attribute	Custom Transformer	AraBART Service
Evaluation setting	In-domain held-out synthetic split	Zero-shot deployment-oriented multi-modal inputs
Model	Character-level seq2seq Transformer	CAMeL-Lab/arabart-qalb15-gec-ged-13
CER	0.0364	0.0950
BLEU	0.292	Not reported in deployment setting
Token-level training accuracy (peak)	91.69%	Not applicable (pre-trained inference service)
Fine-tuning required	Yes (project-specific training)	No
Training time	~100 min (reported GPU run)	None for this study (inference-only integration)
Model size	~50 MB (reported checkpoint size)	~1.5 GB (reported model footprint)
Inference profile	Lightweight checkpoint after training	Heavier pre-trained checkpoint at inference
Supported input modes	Text correction pipeline input	Text, file, OCR image, audio, live speech

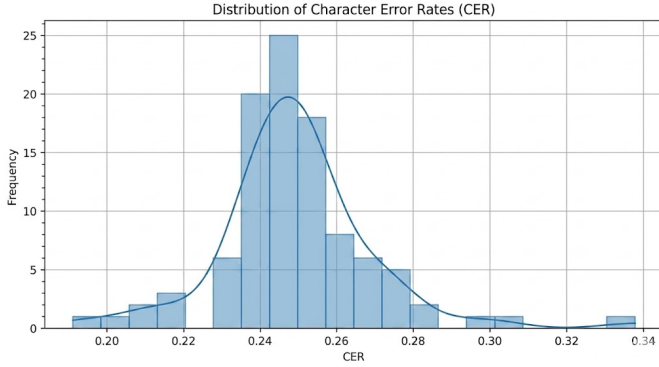


Fig. 3. Distribution of CER over evaluation samples. The central mass around low CER values indicates stable correction behavior for most samples.

misplaced, subsequent characters may drift into shifted token boundaries, increasing local alignment errors. These observations indicate that the current augmentation strategy captures general corruption patterns well but still leaves room for targeted improvement around the most problematic character groups.

Fig. 3 presents the empirical distribution of character error rates across the evaluation samples. The histogram shows that most examples concentrate in the low-CER region, typically between 0 and 0.10, which indicates that the custom Transformer behaves consistently rather than succeeding only on a few isolated cases. The narrow central mass is an important result because it suggests that the model generalizes to most held-out examples with stable correction quality. The longer right tail corresponds to more heavily corrupted samples, where multiple character-level errors accumulate and the model must rely on broader context to recover the intended sequence. Such cases are expected to remain difficult because the problem becomes less constrained as noise density rises.

The shape of the distribution also clarifies the practical reliability of the system. A sharp peak around low error values implies that the model is well aligned with the main data pattern, while the tail indicates that the model still has identifiable failure modes under heavier corruption. This is useful for deployment because it means the model is dependable for most realistic inputs, but it also shows a clear target for future refinement through stronger augmentation, curriculum

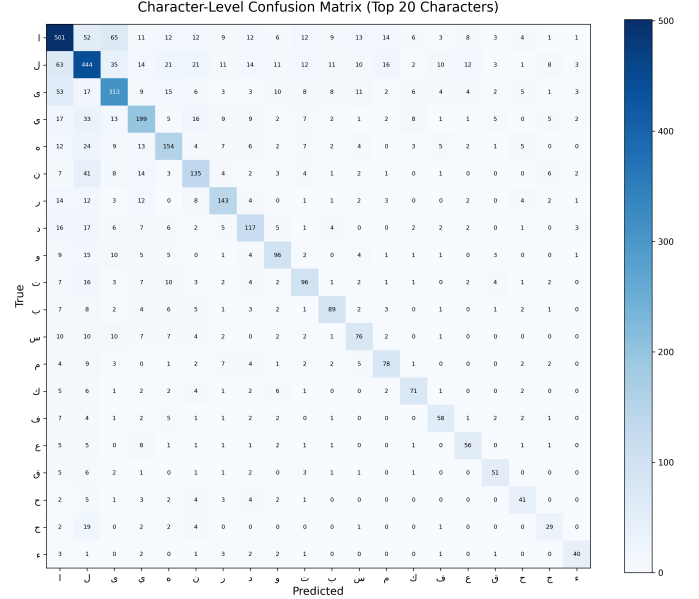


Fig. 4. Character-level confusion matrix (top frequent characters). Dominant off-diagonal regions highlight recurring substitution patterns.

learning, or stronger regularization.

Fig. 4 presents a character-level confusion matrix for the most frequent symbols. The strong diagonal concentration confirms that the model has learned the core identity mapping for the majority of characters, which is the expected baseline for a correction system. The remaining off-diagonal regions are more informative than the diagonal itself because they reveal which substitutions continue to be difficult after training. In this study, the most visible confusions are associated with whitespace and with Arabic letters that are visually or orthographically close. These patterns reinforce the earlier interpretation of the CER distribution and show that the model's remaining errors are not random but structurally concentrated.

The confusion matrix is therefore not only a descriptive figure but also a diagnostic tool. It identifies the most persistent misprediction pairs and suggests where targeted improvements should be applied. For example, synthetic augmentation can focus more on Alef and Hamza contrasts, spacing normalization can be refined for boundary-sensitive cases, and the training objective can be adjusted to penalize specific classes

of confusion more heavily. In this way, the confusion analysis links aggregate evaluation metrics to concrete model failure patterns and provides a practical roadmap for future refinement.

D. Implications for Deployment

Taken together, the results suggest that the project works best as a two-tier correction system: the custom Transformer serves as a compact, domain-aligned model, and AraBART serves as a more fluent deployment layer for different real-world inputs. The error analysis also shows that future improvements should prioritize whitespace handling and Alef/Hamza-style confusions rather than broader architectural expansion. A direct fair-comparison benchmark between both models on a shared external dataset (e.g., QALB) remains an important next experimental step.

VI. CONCLUSION

This paper presented a reproducible Arabic text-correction workflow spanning corpus collection, normalization, synthetic noise generation, model training, evaluation, and deployment. A compact character-level Transformer and a pre-trained AraBART service were developed to represent the two main operating points for Arabic correction: controlled benchmarking and practical user-facing deployment.

The results indicate that the compact model is strong on the project-specific synthetic setting, while the pre-trained service is better suited to fluency-oriented correction across manual text, files, OCR, audio, and live speech. The remaining errors are concentrated in whitespace handling and Alef/Hamza-related confusions, suggesting that future work should focus on targeted augmentation, stronger regularization, earlier stopping, inclusion of classical rule-based baselines, and validation on shared external Arabic correction benchmarks.

REFERENCES

- [1] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, 2014. [Online]. Available: <https://doi.org/10.48550/arXiv.1409.3215>
- [2] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations*, 2015. [Online]. Available: <https://doi.org/10.48550/arXiv.1409.0473>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1706.03762>
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the North American Chapter of the Association for Computational Linguistics*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1810.04805>
- [5] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. [Online]. Available: <https://doi.org/10.18653/v1/2020.acl-main.703>
- [6] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, pp. 1–67, 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.1910.10683>
- [7] W. Antoun, F. Baly, and H. Hajj, "Arabert: Transformer-based model for arabic language understanding," in *Proceedings of the 12th Language Resources and Evaluation Conference*, 2020.
- [8] G. Inoue *et al.*, "The interplay of variant, size, and task type in arabic pre-trained language models," in *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, 2021.
- [9] A. Rozovskaya *et al.*, "The qalb-2014 shared task on automatic text correction for arabic," in *Proceedings of the EMNLP Workshop on Arabic Natural Language Processing*, 2014.
- [10] A. Rozovskaya, N. Habash, R. Roth, N. Zalmout, H. Bouamor, S. Khalifa, O. Obeid, M. Ghoneim, and W. Hassan, "The qalb-2015 shared task on automatic text correction for arabic," in *Proceedings of the BEA Workshop*, 2015.
- [11] G. Abandah, A. Suyyagh, and M. Z. Khedher, "Correcting arabic soft spelling mistakes using bilstm-based machine learning," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 5, 2022. [Online]. Available: <https://doi.org/10.14569/IJACSA.2022.0130594>
- [12] M. Al-Qaraghuli and O. A. Jaafar, "Arabic soft spelling correction with t5," *Jordanian Journal of Computers and Information Technology*, vol. 10, no. 1, 2024. [Online]. Available: <https://doi.org/10.5455/jjcit.71-1699768515>
- [13] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2212.04356>
- [14] OCR.space, "Ocr api documentation," 2024. [Online]. Available: <https://ocr.space/ocrapi>
- [15] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1912.01703>
- [16] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020. [Online]. Available: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: <https://doi.org/10.48550/arXiv.1412.6980>
- [18] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [19] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002. [Online]. Available: <https://doi.org/10.3115/1073083.1073135>
- [20] CAMEL Lab, "Camel-lab/arabart-qalb15-gec-ged-13," Hugging Face Model Card, 2023. [Online]. Available: <https://huggingface.co/CAMEL-Lab/arabart-qalb15-gec-ged-13>